AI Capstone: Milestone 1 Check-in

Project Name: MotionPlay — A Camera-Based Body-Motion Game Controller

Due: Week 3

Focus: Solidifying project scope, understanding the data landscape, planning the technical approach, and establishing a clear project roadmap.

1. Refined Project Charter

1.1 Problem Statement & Goal

Conventional video games are typically controlled via a keyboard, mouse, or hand-held controller. This conventional approach often excludes players who desire a more physical, immersive, or accessible gaming experience. Furthermore, it leaves the rich spatial motion data captured by everyday smartphone cameras entirely unutilized.

- **Goal:** Build an end-to-end system that utilizes a phone camera to capture a player's body pose, recognizes a small, well-defined vocabulary of body gestures using a trained machine learning model, and translates these recognized gestures into game commands in real time to control an on-screen character (with a classic Mario-style platformer as our primary demonstration target).

1.2 Modifications from the Initial Idea

- **Narrowed the Recognition Target:** The initial concept of "recognizing general human actions" proved too broad, as different game genres require vastly different control schemes. We have narrowed the scope to a fixed vocabulary of **5 distinct game-control gestures** (e.g., *Jump, Move-Left, Move-Right, Crouch, and Run/Idle*), which is realistic to train, test, and evaluate within the capstone timeline.
- **Prioritized a Custom-Trained Model over Public Benchmarks:** Because our target gestures are highly specific and require low-latency execution, our core model will be trained on a custom, self-collected, and labeled skeletal dataset. A rule-based heuristic system will be maintained primarily as a baseline reference for comparison.

1.3 Scope Definition

In-Scope	Out-of-Scope
• Capturing a 3D body skeleton using a phone camera	• Recognizing a large, open vocabulary of daily actions
• Building a self-collected, labeled dataset (~5 target gestures)	• Fine-grained finger-level or facial-expression control (v1)
• Training & evaluating an ML gesture/action classifier	• Multi-player or simultaneous multi-user control
• Implementing real-time inference + phone-to-PC data streaming	• Full cross-platform mobile OS support
• Mapping recognized gestures to game input (Key/Controller emu)	• Production-grade / App-Store-ready application
• Developing a working demo controlling a Mario-style game	• VR/AR headset integration
• Quantitative evaluation of model accuracy and latency	• Cloud-hosted, multi-user deployment

2. Data Acquisition & Initial Exploration

2.1 Data Sources Identified & Accessed

While public datasets serve as valuable methodological references, a custom ARKit recording pipeline remains our primary data source due to differences in skeleton topology.

- **Self-Collected Dataset (ARKit Body Tracking)**
    - *Type/Role:* Primary dataset for training, validation, and live demo generation.
    - *Data Format:* CSV sequences containing per-joint 3D positions and rotation quaternions.
    - *Access:* Recorded directly by the project team using an iPhone camera via ARKit.
- **ROSE Lab NTU RGB+D 120**
    - *Type/Role:* Public benchmark dataset for skeleton-based action recognition; utilized as a transfer learning and architectural reference.
    - *Data Format:* RGB videos, depth maps, IR streams, and 25-joint 3D skeleton sequences.
    - *Access:* Academic access request approved via NTU ROSE Lab.

- **Kinetics-Skeleton**
    - *Type/Role:* Large-scale benchmark dataset for unconstrained real-world environments; potential pre-training reference.
    - *Data Format:* 2D skeleton sequences extracted by OpenPose (18 joints + confidence scores).
    - *Access:* Publicly available research dataset via Kinetics GitHub Repository.

2.2 Initial Findings from Sample Data

An exploratory analysis of an initial ARKit sample recording revealed the following structural details and data challenges:

- **Structure:** Data is saved in a long-format CSV structure. Each row contains data for a single joint at a specific frame, including fields for timestamp, frame, joint_name, 3D positions (pos_x, pos_y, pos_z), and rotation quaternions (rot_x, rot_y, rot_z, rot_w).
- **Complexity:** The ARKit skeleton contains **91 joints**, providing highly detailed tracking that covers the full body, both hands (including individual fingers), and facial landmarks. This is significantly denser than NTU RGB+D (25 joints) or Kinetics (18 joints).
- **Observed Anomalies:** Initial data checks revealed irregular timestamp intervals, high noise levels in peripheral finger/facial tracking, inconsistent global orientation across separate recordings, and a clear need for extensive data augmentation to counter the limited volume of manual samples.

2.3 Data Cleaning & Preprocessing Plan

1. **Joint Filtering:** Filter down the 91-joint raw skeleton to a core subset of **20–25 key body joints** (Spine, Head, Shoulders, Arms, Hips, Legs, and Feet), completely stripping out noisy facial and finger data.
2. **Spatial Normalization:** Coordinate transformations will center the coordinate system at the root joint (Hips), scale positions by a reference bone length (to handle user height variations), and rotate the orientation to a canonical front-facing view.
3. **Temporal Resampling:** Resample irregular streams to a uniform frame rate (**30 fps**) and implement a sliding-window technique to generate fixed-length sequences for optimal batch processing.
4. **Dataset Partitioning:** Label each clip with its target gesture; perform a stratified split into Train / Validation / Test sets split *by person* to rigorously test cross-user generalization.
5. **Data Augmentation:** Apply random spatial rotations, small Gaussian jitter, temporal speed scaling, and horizontal left/right mirroring to artificially expand our dataset volume.

2.4 Data Generation Plan (Self-Collected Protocol)

To establish a robust local training set, the team will execute a structured collection protocol:

- **Protocol:** Each of the 4 team members will record every target gesture multiple times under varying conditions (altering speed, camera distance, and starting angles).
- **Target Volume:** 5 gestures × ~200 clips per gesture = **~1,000 labeled clips**, ensuring the model generalizes effectively beyond a single user's movement style.

3. Proposed Technical Approach

3.1 Model Progression Strategy

We adopt a progressive modeling strategy, moving from simple heuristics to state-of-the-art deep learning. This ensures a working pipeline at all times and provides clear performance baselines.

- ▼ **Stage 1: Baseline 1 - Rule-Based**
  - *Technical Implementation*: Manual geometric thresholds (Manual Thresholds)
- ▼ **Stage 2: Baseline 2 - Classical ML**
  - *Technical Implementation*: Traditional machine learning (Random Forest / SVM)
- ▼ **Stage 3: Core ML - Temporal DL (Primary Deliverable)**
  - *Technical Implementation*: Temporal deep learning (1D-CNN / LSTM)
- ▼ **Stage 4: Stretch Goal - Graph NN (Performance Upper Bound)**
  - *Technical Implementation*: Spatio-temporal graph neural networks (ST-GCN / CTR-GCN)

1. **Approach 1: Rule-Based Heuristic (Baseline 1)**

- *Details:* Relies on manually defined geometric thresholds and mathematical conditions derived from raw joint angles and distances.
- *Justification:* Zero training required, extremely low latency, and high interpretability. Serves as our initial end-to-end integration sanity check.

2. **Approach 2: Classical Machine Learning (Baseline 2)**

- *Details:* Models like Random Forest or Support Vector Machines (SVM) trained on hand-crafted features (e.g., angular velocities, inter-joint distances).
- *Justification:* Highly efficient, highly performant on limited dataset volumes, and provides a baseline for ML-driven pattern recognition.

3. **Approach 3: Temporal Deep Learning (Primary Core Model)**

- *Details:* Utilizing 1D Convolutional Neural Networks (1D-CNN) or Recurrent Neural Networks (LSTM/GRU) to handle sequential spatial data over time.

- *Justification:* Capable of naturally capturing the dynamic evolution of a gesture across sequential frames rather than relying on static poses.

4. **Approach 4: Spatio-Temporal Graph Neural Networks (Stretch Goal)**

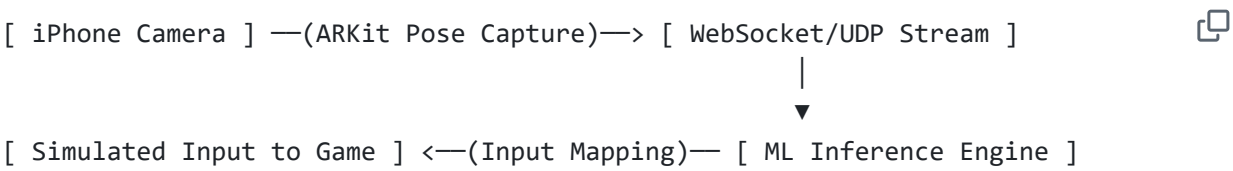
- *Details:* Implementing advanced models such as Spatial-Temporal Graph Convolutional Networks (ST-GCN) or CTR-GCN.
- *Justification:* Treats the human skeleton as a natural graph structure where joints are vertices and bones are edges. Represents the state-of-the-art upper bound for skeleton action recognition.

3.2 Technology Stack

- **Motion Capture:** Swift + Apple ARKit (Native real-time 3D skeleton tracking on iOS).
- **Data & Modeling Pipeline:** Python, NumPy, Pandas, Scikit-learn, and PyTorch (Industry standard for temporal and graph-based deep learning).
- **Real-Time Inference:** PyTorch serving on a local PC (with an option to export to Apple Core ML for future on-device testing).
- **Network Communication:** WebSockets or UDP (OSC protocol) over local Wi-Fi to ensure minimal streaming latency.
- **Input Emulation:** pynput, pydirectinput, or ViGEm (Virtual Gamepad) to simulate keyboard/controller strokes directed into the game engine.
- **Infrastructure & Tools:** Git/GitHub for version control, Jupyter Notebooks for rapid prototyping, and Docker for environment reproducibility.

3.3 System Architecture & Data Flow

> **Operational Modes:**
>
> 1. **Collection Mode:** iPhone (ARKit) ⟶ Stream Pose Data ⟶ PC ⟶ Labeled CSV Storage.
> 2. **Live-Control Mode:** iPhone (ARKit) ⟶ Real-Time Stream ⟶ PC Preprocessing ⟶ ML Model Inference ⟶ Input Mapping ⟶ Game Character Action.



4. Project Plan & Team Roles

4.1 Detailed Task Breakdown (Next 3 Weeks Leading to Milestone 2)

1. **Data & Capture Pipeline:** Finalize the iOS ARKit skeletal recording app, implement the standardized data collection protocol across all team members, and build the initial centralized raw CSV dataset asset.
2. **Preprocessing Implementation:** Code the data cleaning pipeline, including spatial root-centering, bone-length scaling, coordinate rotation, and temporal sliding-window resampling.
3. **System Infrastructure:** Develop and verify the low-latency WebSocket/UDP connection between the iPhone and the PC, and write the virtual keyboard/gamepad simulation script.
4. **Baseline Model Development:** Build and evaluate the Rule-Based heuristic system along with the Classical ML (Random Forest/SVM) baseline classifiers.

4.2 Roles & Responsibilities

Team Member	Core Project Role	Key Responsibilities (Stage 2 Focus)
Kunal Pandey	Data & Capture Lead	Finalize ARKit capture pipeline, enforce recording protocols, and manage dataset assembly.
Lei Li	ML Lead	Architect the preprocessing pipeline (normalization, joint filtering) and structure data split configurations for model training.
Chao Chen	Systems Lead	Engineering the low-latency iPhone-to-PC communication layer, handling streaming, and setting up input emulators.
Chenghua Jiang	Documentation & QA Lead	Monitor dataset balance and quality checks, track validation metrics, and maintain the project repository/documentation.

4.3 Risk Assessment & Mitigation

Identified Risk	Mitigation Strategy
Inconsistent or insufficient training data	Establish rigid pose start/end guidelines, run multiple recording sweeps, and deploy extensive data augmentations (mirroring/scaling).

Identified Risk	Mitigation Strategy
Noisy or unstable ARKit joint tracking	Implement temporal smoothing filters (e.g., moving average or low-pass filters) and drop peripheral joints.
Network latency or connection drops over Wi-Fi	Optimize payload sizes, use lightweight UDP streaming, and profile network performance early in development.
Imbalanced gesture classes	Continuously track dataset distributions via QA dashboarding and targeted recording sessions for minority classes.
Pipeline integration failures	Build decoupled functional modules and validate components independently using mock test data.

5. Questions & Challenges

5.1 Questions for the Instructor

1. Is a self-collected dataset entirely acceptable as our primary data source for evaluation, or does the curriculum strictly require evaluation against a public benchmark like NTU RGB+D?
2. Is a rule-based heuristic system considered a sufficient comparative baseline, provided that our primary deliverable is a trained ML/DL model?
3. Does the evaluation committee prefer an on-device architecture (Core ML on iOS) or a desktop-server architecture (Inference on PC via Wi-Fi streaming), as long as real-time performance metrics are met?
4. What is the grading weight distribution between model classification accuracy, end-to-end system latency, and the functional polish of the live game demo?
5. Is a single game demonstration (e.g., Super Mario Bros platformer) sufficient to satisfy the capstone scope requirements?

5.2 Anticipated Challenges

- **Real-Time Latency Optimization:** Keeping the complete loop (*Capture* ⟶ *Network Stream* ⟶ *Preprocessing* ⟶ *ML Inference* ⟶ *OS Input Emulation*) under **~50ms** to avoid a sluggish player experience.
- **Continuous-to-Discrete Signal Mapping:** Converting a continuous, overlapping stream of body coordinates into crisp, discrete keystrokes without introducing input flicker or accidental double-triggering.

- **Idle State Handling:** Building robust classification boundaries that accurately isolate intentional gesture commands from common random movements or a static "Idle" player state.
- **Cross-User Calibration:** Ensuring invariant accuracy across different players with varied body shapes, clothing types, camera angles, and distance settings.